

3DG ENGINE

Shader Editor Guide

Create, preview, save, and use visual shaders - with practical graph examples for surfaces, post processing, particles, and UI.

Developer Edition

Based on the current OpenGL Shader Editor implementation

July 2026

Includes 11 worked shader graphs, colour-node guidance, material assignment, diagnostics, and a node reference.

How to use this guide

This guide starts with the shortest path to a working shader, then builds toward reusable material parameters and domain-specific effects. Every example uses nodes currently exposed by the editor.

Before you begin

Open the engine editor and enable Panels > Shader Editor. In the Assets panel, enter or create a folder such as Content/Shaders before saving. Shader assets use the .3dgshader extension.

What you will learn

- Choose the correct shader domain and understand its output contract.
- Create nodes by right-clicking or by dragging a pin into empty graph space.
- Use Constant Color, Object Color, and Parameter Color correctly.
- Compile safely, inspect generated GLSL, and recover the last valid shader.
- Assign Surface shaders to materials and objects, Post Process shaders to World Settings, Particle shaders to emitters, and Unlit shaders to HUD widgets.
- Build the eleven example graphs in this guide.

Guide map

Part	Focus	Outcome
1	Editor workflow and graph controls	A compiled and saved shader asset
2	Colour and parameters	Correct fixed, per-object, and material-editable colour
3	Worked examples	Surface, texture, Fresnel, post, particle, and UI effects
4	Integration and troubleshooting	Shaders working in scenes, Play mode, and runtime
5	Node reference	A compact authoring lookup

1. Understand the Shader Editor

The Shader Editor authors the GPU program. Material Maker and the object Inspector author values for parameters exposed by that program. The graph is the source of truth; generated GLSL is derived and can be inspected on the right side of the panel.

The four domains

Domain	Use it for	Important inputs	Where it is assigned
Surface	Lit static or animated geometry	UV, Normal, View Direction, Object Color, parameters	Material Maker or object material
Post Process	Fullscreen camera effects	Scene Color, Depth, Normal, Velocity, Screen UV, Texel Size	World Settings post-process stack
Particle	CPU billboard particle shading	Particle Color, Age, Lifetime, Velocity, Size, Rotation, Frame	Particle Editor / particle asset
Unlit/UI	HUD panels, images, bars, and buttons	Widget UV, Widget Color, Widget Texture, Clip Mask	HUD widget shader path

Choose the domain first

Changing domains replaces the output contract. Domain-specific nodes are validated before compilation, so a Scene Depth node cannot be used in a Surface graph and a Particle Age node cannot be used in a UI graph.

Main areas of the panel

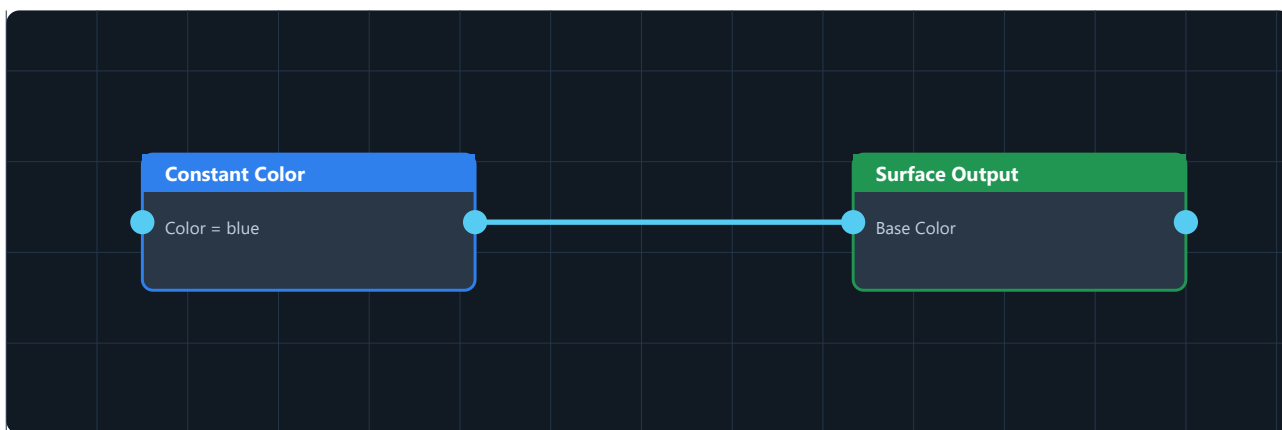
Area	Purpose
Document toolbar	New, Open Selected, Save, Save As, Revert, and Duplicate shader assets.
Compile strip	Compile manually, enable Auto Compile, Apply, or Restore Last Valid.
Typed graph	Place nodes, connect typed pins, pan, zoom, group, copy, and inspect nodes.
Live Preview	Test sphere, cube, plane, or an imported selected model under adjustable lighting.
Generated GLSL	Inspect vertex and fragment source with line numbers and diagnostics.

2. Create your first shader

This first graph produces a fixed blue surface. It is deliberately simple so you can verify the complete workflow before adding parameters or textures.

Step-by-step

- 1 Open Panels > Shader Editor and select New.
- 2 Set Name to SolidBlue and leave Domain as Surface.
- 3 Right-click empty graph space, search for ConstantColor, and create it.
- 4 Select the Constant Color node. In Selected Node, use the Color picker to choose blue.
- 5 Drag the Constant Color result pin to Base Color on Surface Output. You can also release on empty space to open the compatible-node prompt.
- 6 Enable Auto Compile, or select Compile. Confirm the status says Compiled successfully.
- 7 In Assets, enter Content/Shaders, then select Save. The result is SolidBlue.3dgshader.



Example 1 - fixed colour Surface shader

Fast node creation

Drag from a pin and release on empty canvas. The editor opens a searchable list containing only type-compatible nodes for the active domain. Choose a node and it is connected automatically.

Graph editing controls

Action	Control
Add node	Right-click empty graph, or drag a pin to empty graph space
Move node	Left-drag its body
Connect	Drag between compatible pins
Disconnect one pin	Right-click the pin
Node actions	Right-click node for Delete, Duplicate, Copy, or Break All Links
Pan / zoom	Middle-drag / mouse wheel
Undo / redo	Toolbar or Ctrl+Z / Ctrl+Y while Shader Editor is focused
Delete selected	Delete key while Shader Editor is focused, or right-click > Delete Node

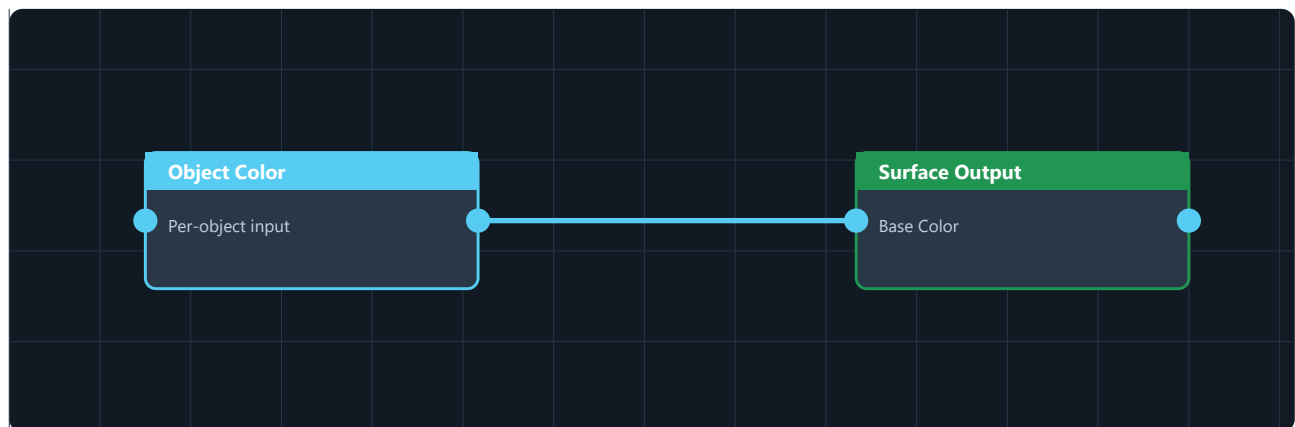
3. Choose the right colour node

The editor provides three useful colour patterns. They look similar in the graph, but they answer different authoring needs.

Node	Where the value comes from	Best use
Constant Color	Stored inside the shader graph	A colour that should be identical everywhere
Object Color	The scene object's Rendering Color / material albedo	One shader reused across differently coloured objects
Parameter Color	A named shader parameter with a default and per-material overrides	Designer-editable materials and instances

Example 2 - per-object colour

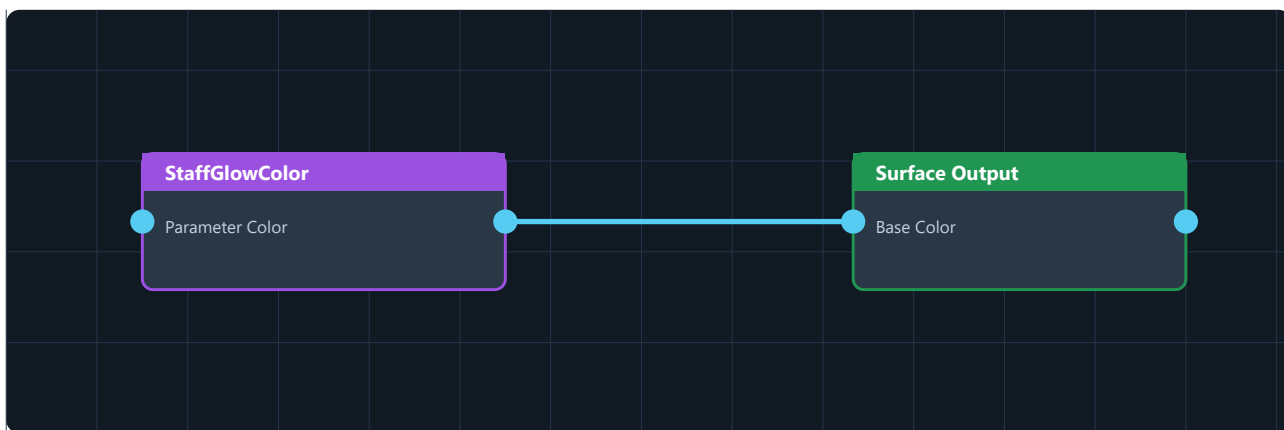
- 1 Create a Surface shader and add ObjectColor.
- 2 Connect Object Color to Surface Output > Base Color.
- 3 Select Object Color and adjust Preview Object Color to test the shader.
- 4 After assignment, select a scene object and change its Rendering Color in the Inspector. Static and animated objects pass that colour to the shader.



Example 2 - one shader, many object colours

Example 3 - material-editable colour

- 1 Add ParameterColor and rename the node to StaffGlowColor.
- 2 Use Default Color in Selected Node to choose the initial colour.
- 3 Connect the result to Base Color and save the shader.
- 4 Assign the shader in Material Maker. The reflected StaffGlowColor value can then be changed without changing the graph.



Example 3 - reusable colour parameter

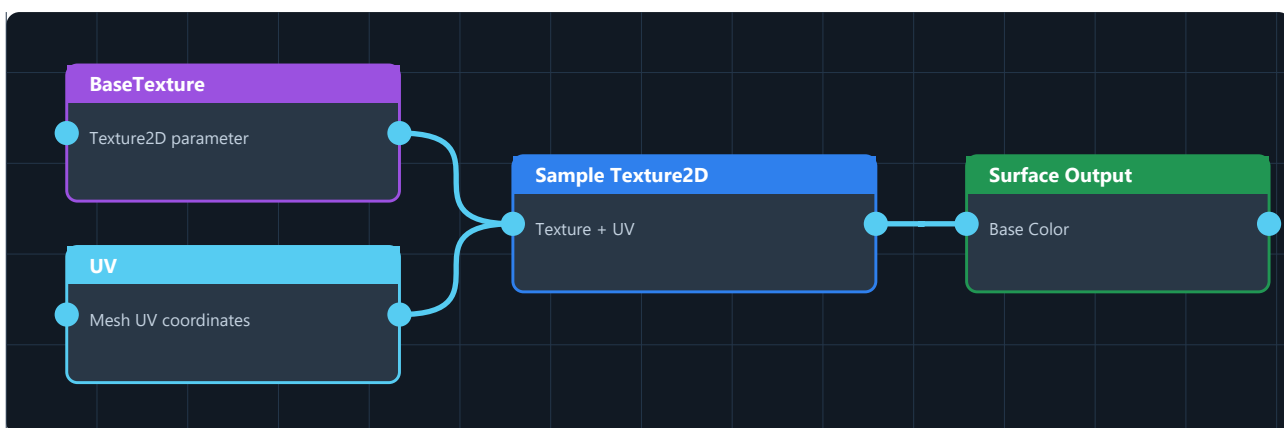
Naming matters

Parameter node names become the exposed material control. Use stable, descriptive names before creating many materials, such as BaseTint, RimWidth, StaffGlowColor, or DamageFlash.

4. Example - texture-based surface shader

This graph samples a Texture2D parameter using the model UV coordinates. It is the standard starting point for a reusable textured surface.

- 1 Create a Surface shader named TexturedSurface.
- 2 Add ParameterTexture2D; rename it BaseTexture.
- 3 Add UV and SampleTexture2D.
- 4 Connect BaseTexture to Sample Texture input A.
- 5 Connect UV to Sample Texture input B.
- 6 Connect Sample Texture result to Surface Output > Base Color.
- 7 Save and assign the shader in Material Maker, then choose the texture for BaseTexture.



Example 4 - texture parameter sampled with mesh UV

Useful variations

- Insert UVTransform before the sampler to apply scale and offset inputs.

- Use ChannelMask after the sampler to extract the red channel as a grayscale output.
- Use NormalMapDecode when a sampled colour represents tangent-space normal data.
- Duplicate the shader asset before experimenting with a production material.

Texture parameter workflow

The shader defines the texture slot. Material Maker or the object Inspector supplies the actual texture path. This keeps the graph reusable across stone, wood, metal, and magical variants.

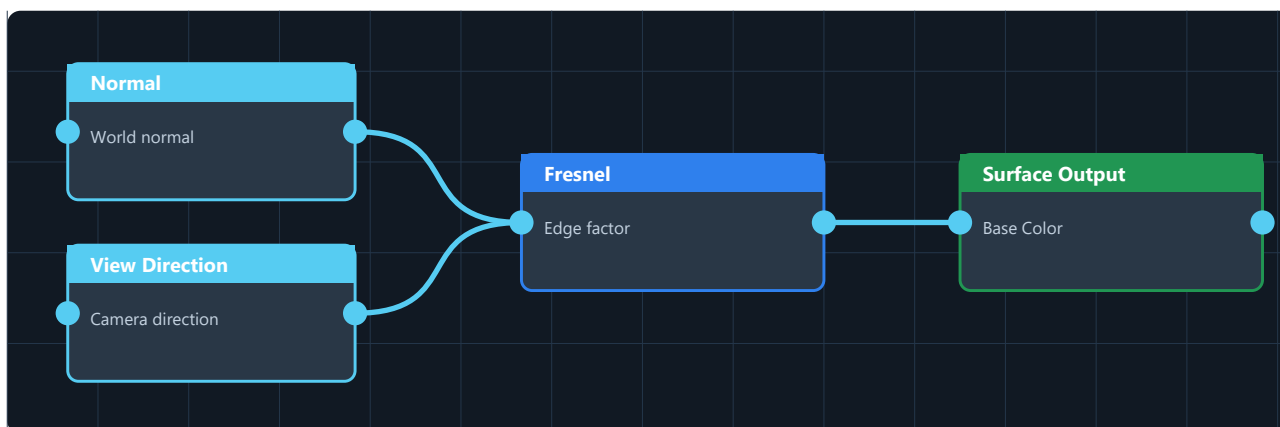
Type notes

Pin type	Typical source	Compatible destination
Texture2D	ParameterTexture2D	SampleTexture2D texture input
Vec2	UV, ScreenUV, ConstantVec2	Sampler UV, UV Transform, Noise
Color / Vec4	Texture sample, Constant Color, Parameter Color	Domain colour output
Float	Noise, depth, age, math	Scalar math or automatic grayscale colour

5. Example - Fresnel rim shader

Fresnel highlights surfaces viewed at a grazing angle. It is useful for magical shields, ghostly objects, selection effects, force fields, and the edge glow of an enchanted staff.

- 1 Create a Surface shader named FresnelRim.
- 2 Add Normal, ViewDirection, and Fresnel.
- 3 Connect Normal to Fresnel input A and View Direction to input B.
- 4 Connect Fresnel result to Base Color. Float-to-colour conversion produces a grayscale rim.
- 5 Preview on a sphere and rotate the camera to inspect the edge response.



Example 5 - grayscale Fresnel rim

Turning it into coloured magic

The current typed graph exposes Fresnel as a Float. The simplest supported result is grayscale. For a coloured production version, use Fresnel as a mask once colour-vector arithmetic is expanded, or combine the Fresnel shader with a coloured object/material workflow. A Parameter Color remains useful for the staff body while a separate Fresnel material handles its aura mesh.

Practical game setup

Place a slightly enlarged transparent aura mesh around the staff crystal. Apply the Fresnel shader to the aura and a Parameter Color shader to the crystal. Spawn particles at the staff socket for fire, frost, or arcane effects.

Related nodes

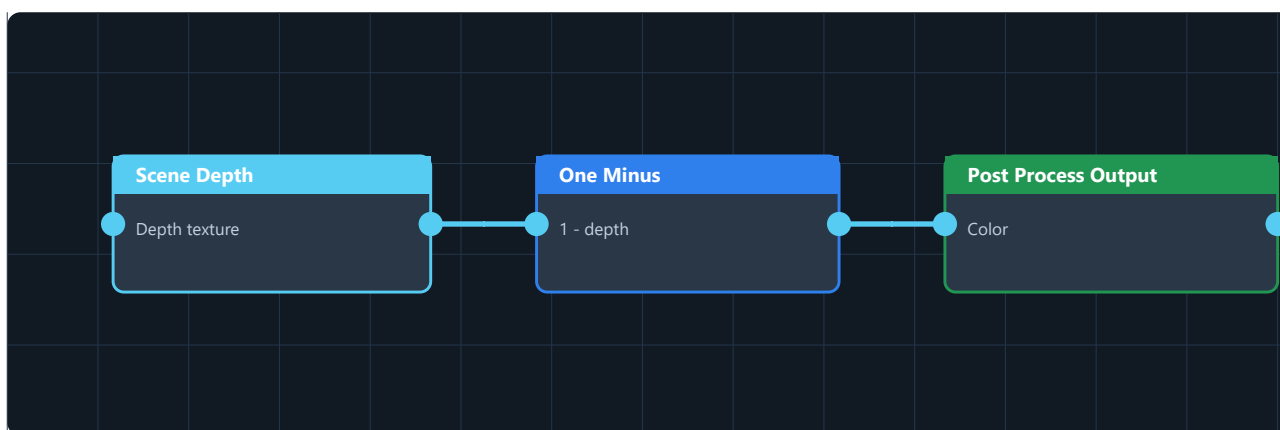
Node	Purpose
Normalize	Makes a vector unit length before directional math.
Dot	Measures directional alignment.
OneMinus	Inverts a scalar mask.
Power	Changes the sharpness of a scalar response.
Saturate	Clamps a float to 0..1.

6. Post Process examples

Post Process graphs operate on the rendered scene. Add them to the ordered effect stack in World Settings. Effects run before bloom, exposure, tone mapping, gamma, and FXAA.

Example 6A - depth inversion

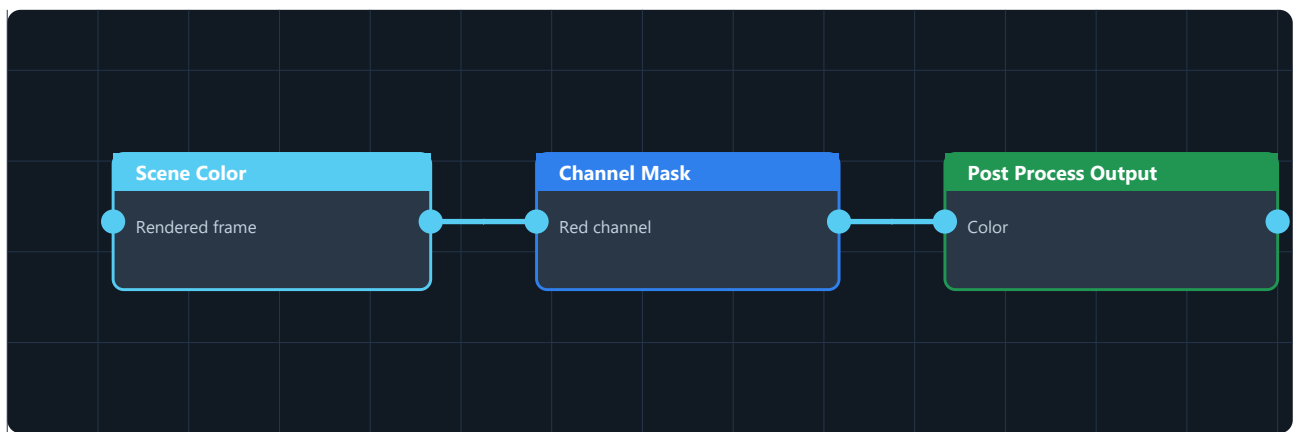
- 1 Create a shader and change Domain to Post Process.
- 2 Add SceneDepth and OneMinus.
- 3 Connect Scene Depth to One Minus, then connect the result to Post Process Output > Color.
- 4 Compile and inspect the preview. Near and far values appear as an inverted grayscale depth image.
- 5 Save as DepthInvert.3dgshader and add it in World Settings.



Example 6A - inverted scene depth

Example 6B - scene channel diagnostic

- 1 Add SceneColor and ChannelMask.
- 2 Connect Scene Color to Channel Mask, then connect the scalar result to Color.
- 3 The current Channel Mask extracts the X/red channel and expands it to grayscale.
- 4 Use this as a debug view when inspecting packed buffers or colour balance.



Example 6B - red-channel diagnostic

Resolution-independent sampling

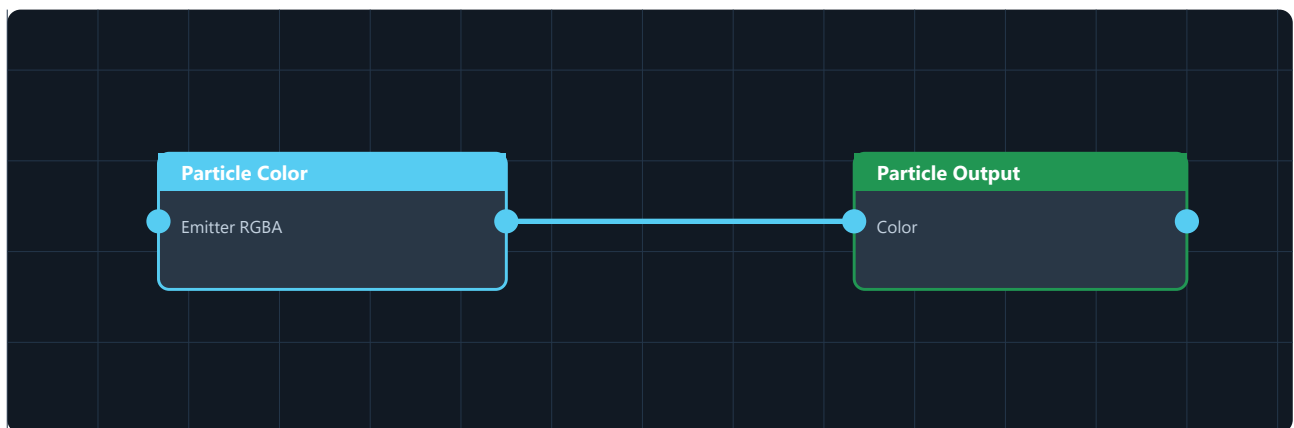
For neighbourhood effects, use Screen UV, Pixel Offset, Texel Size, and explicit sample nodes such as SceneColorSample. Pixel offsets remain consistent when the render resolution changes.

7. Particle shader examples

Particle shaders are assigned in the Particle Editor. Graph-shaded emitters use the CPU billboard path automatically. Unsupported GPU, ribbon, or mesh variants retain their built-in safe renderer.

Example 7A - emitter colour

- 1 Create a shader with Domain set to Particle.
- 2 Add ParticleColor and connect it to Particle Output > Color.
- 3 Save the shader, open the Particle Editor, and assign it under Particle Shader.
- 4 The emitter's colour-over-life and alpha values feed the shader.

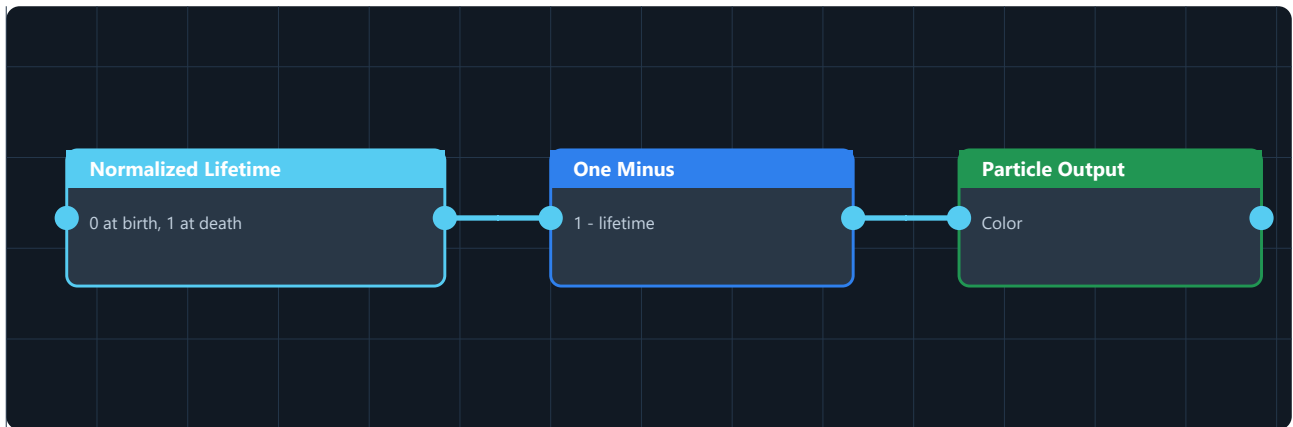


Example 7A - preserve particle colour-over-life

Example 7B - fade using normalized lifetime

- 1 Add NormalizedLifetime and OneMinus.
- 2 Connect lifetime to One Minus and the result to Particle Output.
- 3 At birth the value is near 1; at death it approaches 0.

- 4 The float is expanded to grayscale. Use it as a diagnostic or a starting point for future colour-vector mask arithmetic.



Example 7B - lifetime-driven grayscale fade

Other particle inputs

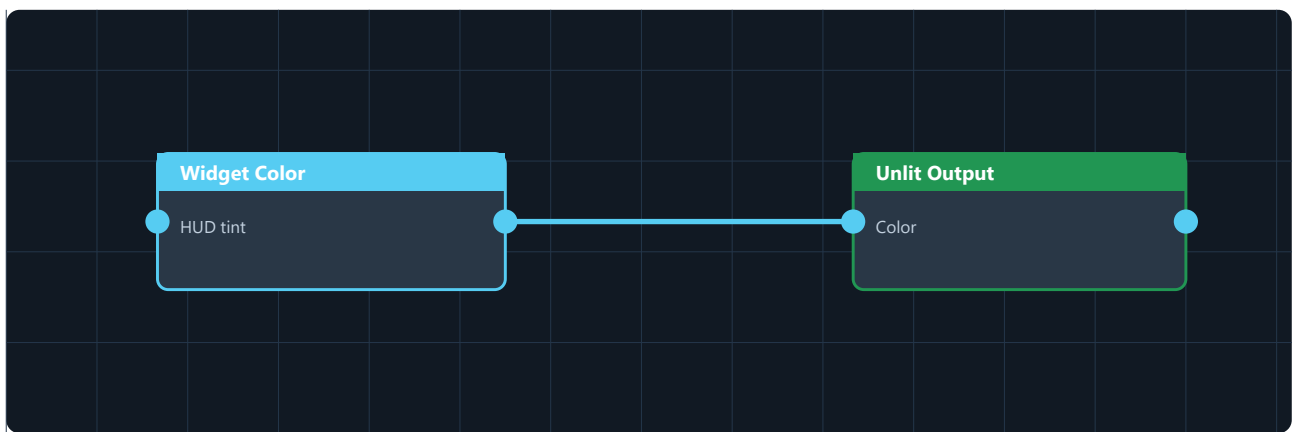
Input	Typical use
Particle Velocity	Direction-aware streaks or colour selection
Particle Size	Size-responsive intensity or debug output
Particle Rotation	Rotation-dependent procedural patterns
Particle Frame	Sprite-sheet frame-aware logic
Trail Coordinates	UV-like coordinates for trail-compatible authoring
Soft Depth	Soft-intersection factor; currently supplied as a safe value

8. Unlit and UI shader examples

Unlit/UI shaders draw HUD rectangles without scene lighting. They can be assigned to panels, images, bars, and buttons. Bitmap text remains on the dedicated text renderer.

Example 8A - widget tint

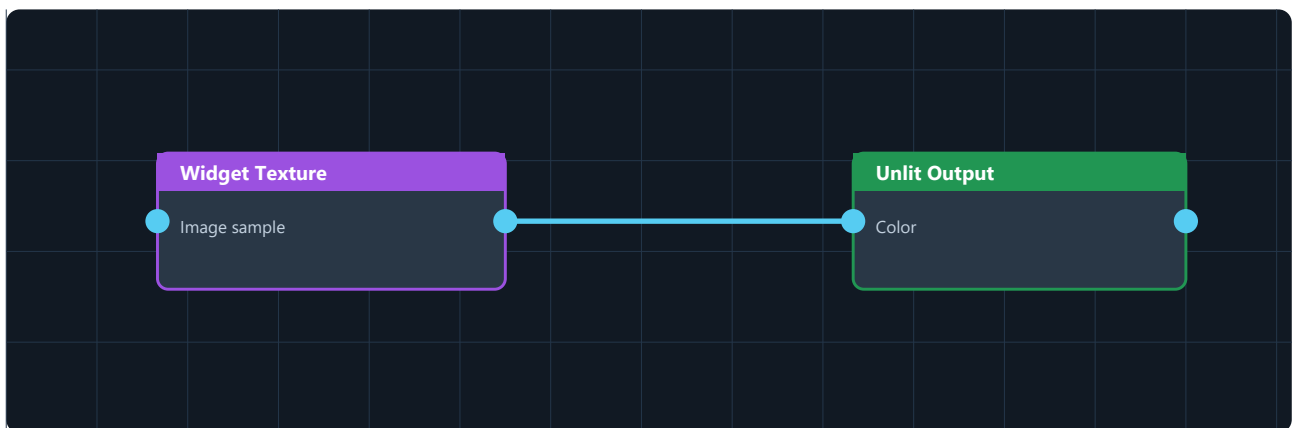
- 1 Create a shader and choose Domain Unlit.
- 2 Add WidgetColor and connect it to Unlit Output > Color.
- 3 Save the shader.
- 4 Open the HUD Editor, select a panel, image, bar, or button, and enter the shader path under Unlit Shader.
- 5 The widget's authored colour or tint becomes the shader input.



Example 8A - widget colour passthrough

Example 8B - widget image

- 1 Add WidgetTexture and connect it directly to Unlit Output.
- 2 Assign the shader to an Image widget that has an image asset.
- 3 The widget texture is sampled using Widget UV by the domain template.



Example 8B - image widget shader

UI batching rule

Custom UI shaders use isolated rectangle draws and then restore the bitmap-font state. This prevents a custom panel or button shader from corrupting later HUD text.

UI helper nodes

Node	Meaning
WidgetUV	Normalized rectangle coordinates
WidgetColor	Widget-authored RGBA colour
WidgetTexture	Optional widget image sample
ClipMask	Clipping helper; current safe result is 1
SignedDistance	Distance from a Vec2 input, useful for procedural shapes

9. Save, assign, and reuse shaders

A shader is only one layer of the content workflow. The following routes connect each domain to the system that supplies its parameters and render context.

Surface assignment

- 1 Save the shader in Content/Shaders.
- 2 Open Material Maker and assign the selected .3dgshader to a material.
- 3 Edit reflected parameters and texture slots, then save the .3dgmater.
- 4 Select a scene object and assign the material in the Inspector.
- 5 Use object parameter overrides when one instance should differ from the source material.

Other domains

Domain	Assignment path	Persistence
Post Process	World Settings > post-process effects	Scene and runtime export preserve order, enabled state, and values
Particle	Particle Editor > Particle Shader	Particle asset version 12 preserves shader and parameters
Unlit/UI	HUD Editor > selected widget > Unlit Shader	HUD asset version 2 preserves shader and parameters

Compile controls

Control	Use
Compile	Generate and compile immediately.
Auto Compile	Recompile after graph edits.
Apply	Compile and mark the current valid program for use.
Restore Last Valid	Return to the last successfully generated source after an invalid edit.
Revert	Reload the saved graph from disk.
Duplicate	Save a uniquely named copy with a new asset identity.

Safe failure behaviour

An invalid shader does not replace the last working program. Missing or invalid domain shaders have visible fallbacks, so an object should not silently disappear.

Runtime consistency checklist

- Save both the shader and its material or particle/HUD asset.
- Save the scene after assignment.
- Enter Play mode and compare against Edit mode.
- Export and validate the runtime scene when preparing a standalone build.
- Keep shader and texture assets under the project Content root for relocatable paths.

10. Troubleshooting

Symptom	Likely cause	Resolution
No connection can be made	Pin types are incompatible	Drag into empty space and choose from the filtered compatible-node prompt.
Graph reports a domain error	A node belongs to another domain	Replace it with an input from the active domain.
Preview is magenta	Compilation or asset loading failed	Read the first diagnostic, select it to focus the node/source line, then restore last valid if needed.
Object Color is white or unexpected	The scene object's Rendering Color/material albedo is white	Set Preview Object Color for preview, then set Rendering Color on the scene object.
Parameter Color is black	No default or override is supplied	Select the node and set Default Color; confirm the material reflects the parameter.
Texture appears white	Texture parameter has no assigned texture or UV is missing	Assign the texture in Material Maker and connect UV to SampleTexture2D.
Delete removes a scene object	Focus is outside Shader Editor	Click the graph first or right-click the node and choose Delete Node.
Particle shader switches backend	Custom graph shading is CPU billboard only	This is intentional; built-in paths remain for unsupported variants.
Scene effect does not run	Post Process shader was not added/enabled	Add it to World Settings, enable it, and verify stack order.

Reading diagnostics

- The status strip shows compile success, duration, active variant, parameter count, texture count, and cache state.
- Diagnostics preserve shader stage and generated line information.
- Selecting a diagnostic focuses the responsible graph node when line-to-node mapping is available.
- Use the Vertex and Fragment tabs to inspect numbered generated GLSL.
- Cycles, duplicate IDs, missing required inputs, invalid types, and excessive resources are rejected before unsafe runtime use.

A useful debugging sequence

1. Disconnect the last node.
2. Compile.
3. Reconnect one branch at a time.
4. Inspect the generated Fragment source.
5. Restore Last Valid if the preview remains incorrect.

11. Compact node reference

This table groups the nodes currently offered by the create-node palette. Domain filtering hides incompatible domain-specific inputs when a wire is dropped on empty space.

Category	Nodes
Common inputs	UV, WorldPosition, LocalPosition, Normal, Tangent, ViewDirection, CameraPosition, ObjectColor, VertexColor, Time, DeltaTime
Parameters	ParameterFloat, ParameterInt, ParameterBool, ParameterVec2, ParameterVec3, ParameterVec4, ParameterColor, ParameterTexture2D
Constants	ConstantFloat, ConstantVec2, ConstantColor
Math	Add, Subtract, Multiply, Divide, Min, Max, Clamp, Saturate, Power, SquareRoot, Absolute, Sign, Floor, Fraction, Modulo
Vector	Compose, Split, Swizzle, Dot, Cross, Normalize, Length, Reflect, Lerp
Texture	SampleTexture2D, NormalMapDecode, UVTransform, ChannelMask
Utility	OneMinus, Remap, Smoothstep, Fresnel, Noise, Comparison, Select
Post Process	ScreenUV, SceneColor, SceneDepth, SceneNormal, SceneVelocity, TexelSize, Exposure, explicit sample nodes, PixelOffset
Particle	ParticleColor, ParticleAge, NormalizedLifetime, ParticleVelocity, ParticleSize, ParticleRotation, ParticleFrame, TrailCoordinates, SoftDepth
Unlit/UI	WidgetUV, WidgetColor, WidgetTexture, ClipMask, SignedDistance

Value types

Type	Meaning	Example
Float	One scalar number	Rough mask, age, noise
Int	Whole number	Mode or index
Bool	True/false condition	Comparison or Select condition
Vec2	Two-number vector	UV coordinates
Vec3	Three-number vector	Position, normal, direction
Vec4 / Color	Four-number vector	RGBA colour
Texture2D	Sampleable image resource	Albedo or mask texture

Current output scope

The current visual compiler primarily authors the domain Color result. Surface lighting is applied by the domain template. Material properties such as metallic and roughness remain available through the material system while broader vector arithmetic and additional authored outputs can be expanded in future versions.

12. Shader ideas to build next

Use these as small exercises after completing the worked examples.

Idea	Domain	Suggested graph
Procedural stone speckle	Surface	UV -> Noise -> Base Color
Object-coloured team material	Surface	Object Color -> Base Color
Designer-controlled crystal	Surface	Parameter Color -> Base Color
UV texture viewer	Surface	Texture parameter + UV -> SampleTexture2D -> Base Color
Normal diagnostic	Surface	Normal -> Split -> Base Color (grayscale X component)
Depth debug camera	Post Process	Scene Depth -> Color
Inverted depth fog mask	Post Process	Scene Depth -> One Minus -> Color
Velocity magnitude view	Post Process	Scene Velocity -> Length -> Color
Particle age debug	Particle	Normalized Lifetime -> Color
Particle fade mask	Particle	Normalized Lifetime -> One Minus -> Color
HUD colour passthrough	Unlit	Widget Color -> Color
HUD texture passthrough	Unlit	Widget Texture -> Color
Procedural circular widget	Unlit	Widget UV -> Signed Distance -> Color

Recommended learning order

- 1 Solid Constant Color
- 2 Object Color
- 3 Parameter Color and a saved material
- 4 Texture sampling with UV
- 5 Noise or Fresnel scalar effects
- 6 Post Process depth diagnostic
- 7 Particle colour and lifetime
- 8 HUD widget colour and texture

Final rule of thumb

Use a Constant when the value belongs to the shader, an Object input when it belongs to each scene object, and a Parameter when a material designer should edit it.

End of guide

This document reflects the Shader Editor implementation in the current 3DG Engine workspace as of July 2026.